

大作业实验报告

路柠泽 2022010076

一、项目介绍

作业实现了一款基于 Python 与 Tkinter 的关灯游戏桌面程序。游戏在 $N \times N$ (N 为 3、5、7) 的方形灯阵上进行：每盏灯只有「亮」与「灭」两种状态，玩家通过点击格子触发翻转规则，目标是将所有灯熄灭。在“自定义关卡”中，玩家需通过“十字翻转”或“X 形翻转”规则处理不同难度的残局。

二、环境与架构

本项目只依赖 Python 标准库，请确保 Python 3 已安装且 tkinter 可用。运行 src 目录下的 main.py 即可启动程序。

文件结构

路柠泽_2022010076/

```
|—— src/
|   |—— main.py    # 程序入口
|   |—— logic.py   # 游戏逻辑
|   |—— ui.py      # 图形界面
|—— report.pdf     # 实验报告
|—— demo.mp4      # 演示视频
```

项目采用分离架构，其中 **main.py** 为入口，调用 `app()` 启动程序。**ui.py** 为图形用户界面，负责窗口布局、用户交互、计时和结算等功能。**logic.py** 为游戏核心逻辑，负责翻转的规则、胜负判断和关卡生成。通过用户操作→**ui.py** 处理事件→调用 **logic.py** 修改棋盘→**ui.py** 刷新界面的流程实现任务。

三、功能实现

主要流程

初始化：`app()` → `ui()` 创建窗口 → `change_mode()` 修改设置 → `reset()` 重置当前游戏 → `grid()` 生成按钮 → `create()` 或 `custom()` 生成棋盘 → `refresh()` 同步界面。

对局：`click()` → 首次点击 `start_timer()` → `flip()` 翻转 → `refresh()` 更新显示 → `off()` 判定是否全灭 → 全灭则 `win()` 弹窗。

改设置：修改模式/大小/规则/难度 → `change_mode()` → 未对局则 `reset()` 重建；对局中则 `change_control()` 锁定控件。

翻转是游戏的核心操作：`flip(plate, x, y, M)` 根据当前规则 M 确定需要翻转的格子集合，十字翻转时翻转被点击格子及其上下左右四个相邻格，X 形翻转时则翻转被点击格子及其四个对角相邻格，每次翻转前都会对坐标做边界判断，只有满足 $0 \leq h < N$ and $0 \leq v < N$ 的格子才会被加入翻转列表，从而避免点击角落或边缘时出现数组越界。确定翻转范围后，程序对这些格子的布尔状态逐个取反，True 表示灯亮，False 表示灯灭。胜负判定由 `off(plate)` 完成，它遍历整个棋盘，当所有格子均为 False 时返回真，表示玩家已经通关。

经典模式下，程序通过 `create(N)` 生成一个 $N \times N$ 的全亮棋盘，并固定使用十字翻转规则。玩家可以在界面上选择 3×3 、 5×5 或 7×7 的棋盘大小，首次点击任意方块后开始计时并锁定设置项，此后每次点击都会调用 `flip` 更新 `plate`，再通过 `refresh()` 把逻辑状态同步到按钮颜色上，同时更新操作次数与游戏时长。当检测到棋盘全灭时，程序调用 `win()` 停止

计时，弹出胜利窗口展示本局步数和用时，然后自动重置进入下一局。对于 3×3 经典模式，已知解法为依次点击四个角和中心格，即可在较短时间内完成演示。

自定义关卡模式采用作业要求的逆向模拟法生成初始局面。程序先从全灭棋盘出发，`custom()` 根据玩家选择的难度确定随机翻转次数 T ：简单难度 T 为 1，中等难度 T 在 2 到 3 之间随机，困难难度 T 在 4 到 9 之间随机。随后从全部坐标中随机抽取 T 个不重复位置，按照当前选定的翻转规则依次执行 `flip`，得到亮灭混合的初始残局。由于这些残局是由合法翻转操作从全灭状态构造出来的，因此必然有解，且最优解步数就是生成时使用的 T 步。自定义模式下玩家仍可选择棋盘大小，并可选择十字翻转或 X 形翻转；界面通过单选按钮在经典与自定义之间切换，当处于经典模式时，规则与难度相关控件会被禁用，当切换到自定义模式后才开放这些选项。一局游戏开始后程序会禁用全部设置控件，禁止中途切换翻转规则，以保证当前局的逻辑一致性。

界面部分由 `ui()` 统一构建，顶部为标题，中部为设置区，包括游戏模式、棋盘大小、翻转规则和难度等控件，下方为操作次数与游戏时长的信息栏，再下方是灯阵按钮区域和重新开始按钮。灯的显示通过 `change_light()` 实现，亮灯使用宝蓝色，灭灯使用深灰色，两者对比明显，便于辨认。下拉框由 `add_combo()` 统一创建，它把用户看到的文字选项映射为程序内部使用的数值或常量，选中后更新对应变量并触发 `change_mode()`，若当前不在对局中则调用 `reset()` 按新设置重建棋盘。计时功能在玩家第一次点击方块时启动，`start_timer()` 记录开始时间并将 `playing` 置为真，随后 `timerloop()` 在对局期间周期性刷新用时显示；步数则在每次有效点击后累加。若玩家点击重新开始，或修改了尚未开局时的设置，程序都会通过 `reset()` 清零步数和计时，重新生成棋盘并刷新界面。

在鲁棒性方面，程序在逻辑和交互上都进行了防护。除了 `flip` 内部的边界检测外，`click()` 在处理点击时也会再次检查行列是否落在合法范围内。创建按钮时使用 `lambda x=x, y=y: click(x, y)` 的写法，避免循环变量被错误绑定到同一个坐标。对局进行中若用户尝试修改模式、大小或规则，界面只会锁定控件而不会破坏当前棋盘；若首次点击时发现棋盘为空或已经全灭，则会先重置再开始，避免出现非法状态。综合这些处理，程序在频繁点击边缘格子或快速连续操作时能够保持稳定，不会出现崩溃或越界异常。

功能测试与演示请见视频。